

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

https://github.com/seejokim1/NA_with_python

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
4	수치미분 (Numerical Differentiation)	4
5	수치적분 (Numerical Integration)	5
6	수치 보간 (Interpolation)	6
6.1	수치 보간 개요	6
6.1.1	보간과 곡선 근사의 머신러닝 관점	6
6.2	뉴턴 분할 차분법	7
6.2.1	Newton의 차분 (Divided-Difference) 1차 선형 보간	7
6.2.2	Newton의 차분(divided-difference) 2차 보간	8
6.2.3	Newton의 차분(divided-difference) n 차 보간	8
6.2.4	Newton의 차분(divided-difference) 예제	9
6.3	Lagrange Interpolation 수치 보간	10
6.3.1	Lagrange Interpolation 정의	10
6.3.2	Lagrange Interpolation 예제	12
6.4	스플라인 보간	13
6.4.1	Quadratic Spline	13
6.4.2	Quadratic Spline 예제	14
6.4.3	Cubic Spline	14
6.4.4	Cubic Spline 예제	14
6.5	수치보간법들의 차이점	15
6.5.1	뉴턴의 분할 차분(Newton's Divided Difference Method)과 라그랑주 보간법(Lagrange Interpolation)의 동일함	15
6.5.2	Cubic Spline이 다른 보간법과 다른 이유	16
6.5.3	코드 구현	16
6.6	연습문제	16

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

Chapter 4

수치미분 (Numerical Differentiation)

Chapter 5

수치적분 (Numerical Integration)

Chapter 6

수치 보간 (Interpolation)

6.1 수치 보간 개요

6.1.1 보간과 곡선 근사의 머신러닝 관점

1. 보간(Interpolation)의 기본 개념 보간은 주어진 데이터 점 (x_i, y_i) 를 모두 통과하는 곡선을 찾는 방법이다. 데이터 사이의 값을 예측하기 위해 사용되며, 수치해석과 데이터 분석의 기본 기법이다.

- 주어진 데이터 포인트를 정확히 통과
- Newton, Lagrange 다항식, Spline 보간 등이 대표적
- 공학 계산 및 과학적 모델링에 활용

2. 곡선 근사(Approximation) 곡선 근사는 모든 점을 반드시 통과하지 않아도 되며, 전체 데이터의 오차를 최소화하는 곡선을 찾는다.

- 최소제곱법(Least Squares)
- 회귀(Regression)
- 노이즈가 포함된 데이터에 적합

3. 머신러닝과의 관계 머신러닝은 보간 및 곡선 근사의 확장된 형태로 볼 수 있다.

- 데이터로부터 패턴을 학습하여 함수 모델 생성
- 과적합(Overfitting) 문제 존재
- 구간별(piecewise) 모델링이 효과적일 수 있음

4. 활용 도구 Python 환경에서 다음 라이브러리들이 활용된다.

- NumPy
- SciPy
- TensorFlow

5. 핵심 정리

- 보간: 모든 데이터 점을 정확히 통과
- 근사: 오차 최소화
- 머신러닝: 대규모 데이터 기반 확장 모델링
- Spline 보간은 부드러운 결과 제공

6.2 뉴턴 분할 차분법

6.2.1 Newton의 차분 (Divided-Difference) 1차 선형 보간

Newton 다항식은 차분(divided difference)을 이용하여 데이터 점들을 통과하는 보간 다항식을 구성하는 방법이다. 1차 보간은 두 점을 연결하는 직선으로 근사하는 방법이다.

1. 선형 보간의 정의

두 점 (x_0, f_0) , (x_1, f_1) 가 주어졌을 때, 선형 보간 다항식은 다음과 같다.

$$p_1(x) = b_0 + b_1(x - x_0)$$

여기서

$$b_0 = f(x_0)$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

즉, b_1 은 두 점 사이의 기울기이다.

2. Newton 차분 표현

1차 Newton 차분은 다음과 같이 정의된다.

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

따라서 1차 Newton 보간식은

$$p_1(x) = f(x_0) + f[x_0, x_1](x - x_0)$$

3. 특징

- 데이터 두 점을 정확히 통과한다.
- 기울기는 평균 변화율과 동일하다.
- 고차 보간의 기초가 되는 가장 단순한 Newton 형태이다.
- 데이터가 추가되면 차분 항을 확장하여 다항식을 구성할 수 있다.

6.2.2 Newton의 차분(divided-difference) 2차 보간

Newton의 2차 보간은 세 점 (x_0, f_0) , (x_1, f_1) , (x_2, f_2) 를 통과하는 2차 다항식을 구성하는 방법이다.

1. 2차 보간 다항식

$$p_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) \quad (6.2.2-1)$$

2. 계수 계산

$$b_0 = f(x_0)$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0} \quad (6.2.2-2)$$

또는 분할차분 기호를 사용하면

$$b_0 = f[x_0]$$

$$b_1 = f[x_0, x_1]$$

$$b_2 = f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} \quad (6.2.2-3)$$

3. 분할차분 형태의 일반 표현

$$p_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \quad (6.2.2-4)$$

4. 특징

- 주어진 점을 정확히 통과하는 2차 다항식을 구성한다.
- 점이 하나 추가될 때마다 차수가 1 증가한다.
- 분할차분 형태는 고차 보간으로 자연스럽게 확장 가능하다.

6.2.3 Newton의 차분(divided-difference) n 차 보간

Newton 보간은 분할차분을 이용하여 $n + 1$ 개의 점 $(x_0, f_0), \dots, (x_n, f_n)$ 을 지나는 n 차 다항식을 구성하는 방법이다.

1. 고차 분할차분 정의

$$f[x_0, x_1, \dots, x_n] = \frac{f[x_1, x_2, \dots, x_n] - f[x_0, x_1, \dots, x_{n-1}]}{x_n - x_0} \quad (6.2.3-1)$$

이 정의를 반복하면 임의의 차수까지 확장할 수 있다.

2. Newton n 차 보간 다항식

$$f(x) = P_n(x)$$

$$\begin{aligned} P_n(x) = & f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots \\ & + (x - x_0)(x - x_1) \cdots (x - x_{n-1})f[x_0, x_1, \dots, x_n] \end{aligned} \quad (6.2.3-2)$$

3. 일반 합 형태

$$P_n(x) = \sum_{i=0}^n f[x_0, x_1, \dots, x_i] \prod_{j=0}^{i-1} (x - x_j) \quad (6.2.3-3)$$

4. 특징

- 점이 추가되면 새로운 항만 추가하면 된다.
- 고차 보간으로 자연스럽게 확장 가능하다.
- 계산 과정이 체계적이며 수치해석에서 널리 사용된다.

6.2.4 Newton의 차분(divided-difference) 예제

본 예제는 Newton의 분할차분(Newton's Divided Difference) 방법을 사용하여 주어진 데이터 점들을 보간하고 결과를 시각화하는 과정을 다룬다.

1. 데이터 설정

$$x = [0, 1, 2, 3], \quad y = [1, 2, 0, 5]$$

2. 분할차분 계산

주어진 데이터에 대해 분할차분 계수를 계산하여 Newton 보간 다항식의 계수를 구한다.
결과 계수:

$$[1.0, -1.5, 1.1667, \dots]$$

3. 보간 다항식 구성

Newton 보간 다항식은 다음 형태로 구성된다.

$$P_n(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots$$

4. 새로운 점에서의 계산

새로운 입력 값 x_{new} 에 대해 Newton 다항식을 이용하여 y_{new} 를 계산한다.

5. 시각화

- 원래 데이터 점과 보간 다항식을 함께 플롯한다.
- 보간 결과와 실제 점의 차이를 시각적으로 비교한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter06/section_06_02_04_newton_divided_difference.py

```
import numpy as np

def divided_diff(x, y):
    n = len(y)
    coef = np.zeros((n,n))
    coef[:,0] = y

    for j in range(1,n):
        for i in range(n-j):
            coef[i,j] = (coef[i+1,j-1] - coef[i,j-1]) / (x[i+j] - x[i])

    return coef[0,:]
```

6.3 Lagrange Interpolation 수치 보간

6.3.1 Lagrange Interpolation 정의

라그랑주 보간은 주어진 $n + 1$ 개의 데이터 점

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

을 정확히 지나는 보간 다항식 $P_n(x)$ 을 구성하는 방법이다.

1. 기본 정의

기저 다항식(Lagrange basis polynomial) $L_i(x)$ 는 다음 조건을 만족한다:

$$L_i(x_j) = \begin{cases} 1 & (i = j) \\ 0 & (i \neq j) \end{cases}$$

보간 다항식은

$$P_n(x) = \sum_{i=0}^n y_i L_i(x) \quad (6.1)$$

으로 정의된다.

기저 다항식의 일반형은

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \quad (6.2)$$

이다.

—

2. 선형 보간 (n=1)

두 점 $(x_0, y_0), (x_1, y_1)$ 에 대해

$$L_0(x) = \frac{x - x_1}{x_0 - x_1}, \quad L_1(x) = \frac{x - x_0}{x_1 - x_0}$$

따라서 보간 다항식은

$$P_1(x) = y_0 L_0(x) + y_1 L_1(x) \quad (6.3)$$

3. 2차 보간 (n=2)

세 점 $(x_0, y_0), (x_1, y_1), (x_2, y_2)$ 에 대해

$$P_2(x) = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x)$$

각 기저 다항식은

$$L_0(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)}$$

$$L_1(x) = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)}$$

$$L_2(x) = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

4. 일반 n차 보간

$$P_n(x) = \sum_{i=0}^n y_i \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \quad (6.4)$$

핵심 특징

- 모든 데이터 점을 정확히 통과한다.
- 차수는 데이터 개수보다 1 작다.
- 전역(global) 다항식 보간 방법이다.
- 고차가 되면 Runge 현상이 발생할 수 있다.

6.3.2 Lagrange Interpolation 예제

다음 데이터 점이 주어졌다고 하자.

$$x = [0, 1, 2, 3], \quad y = [1, 2, 0, 5]$$

즉,

$$(0, 1), (1, 2), (2, 0), (3, 5)$$

을 모두 지나는 3차 보간 다항식 $P_3(x)$ 를 구한다.

1. 보간 다항식 공식

라그랑주 보간 다항식은

$$P_3(x) = \sum_{i=0}^3 y_i L_i(x)$$

여기서

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^3 \frac{x - x_j}{x_i - x_j}$$

2. 각 기저 다항식

$$L_0(x) = \frac{(x-1)(x-2)(x-3)}{(0-1)(0-2)(0-3)}$$

$$L_1(x) = \frac{(x-0)(x-2)(x-3)}{(1-0)(1-2)(1-3)}$$

$$L_2(x) = \frac{(x-0)(x-1)(x-3)}{(2-0)(2-1)(2-3)}$$

$$L_3(x) = \frac{(x-0)(x-1)(x-2)}{(3-0)(3-1)(3-2)}$$

3. 최종 보간 다항식

$$P_3(x) = 1 \cdot L_0(x) + 2 \cdot L_1(x) + 0 \cdot L_2(x) + 5 \cdot L_3(x)$$

이를 전개하면 3차 다항식이 된다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter06/section_06_03_02_lagrange_interpolation.py

```
def lagrange(x_data, y_data, x):
    total = 0
    n = len(x_data)
    for i in range(n):
        term = y_data[i]
        for j in range(n):
            if j != i:
                term *= (x - x_data[j]) / (x_data[i] - x_data[j])
        total += term
    return total
```

6.4 스플라인 보간

6.4.1 Quadratic Spline

주어진 데이터 포인트 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ 에서 2차 스플라인 보간은 각 데이터 점 사이를 2차 다항식으로 연결하여 곡선을 생성하는 방법이다.

구간 $[x_i, x_{i+1}]$ 에 대해 2차 스플라인은 다음과 같이 정의된다.

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 \quad (6.4.1-1)$$

조건

1. 데이터 점 통과

$$S_i(x_i) = y_i, \quad (6.5)$$

$$S_i(x_{i+1}) = y_{i+1} \quad (6.4.1-2)$$

2. 1차 미분 연속

$$S'_i(x_{i+1}) = S'_{i+1}(x_{i+1}) \quad (6.4.1-3)$$

3. 자연 경계 조건 (Natural Boundary Condition)

양 끝점에서 2차 도함수가 0이 되도록 설정한다.

$$S''_0(x_0) = S''_{n-1}(x_n) \quad (6.4.1-4)$$

아래 그림은 Quadratic Spline 보간법을 사용하여 데이터 포인트를 부드럽게 연결한 결과이다. 데이터 포인트는 초록색 점으로 표시되었으며, 주황색 곡선이 2차 스플라인 곡선이다. 각 구간에는 $a_0, b_0, c_0, \dots$ 계수가 적용된다.

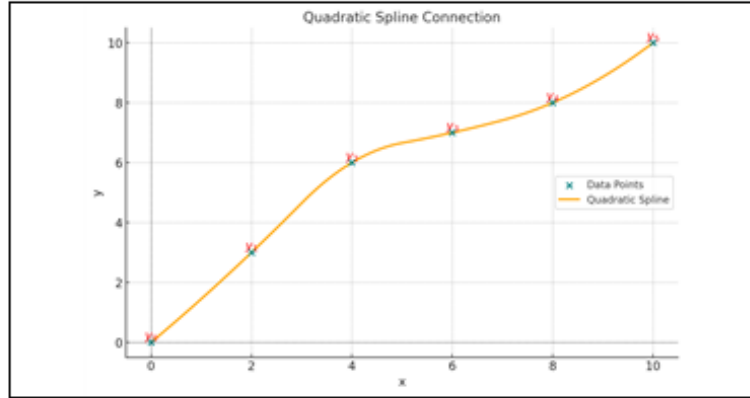


Figure 6.1: Quadratic Spline Connection

6.4.2 Quadratic Spline 예제

이 코드는 Quadratic Spline Interpolation(2차 스플라인 보간)을 사용하여 데이터를 부드럽게 연결한 후, 결과를 시각화하는 과정을 보여준다. Quadratic Spline은 각 구간을 2차 다항식으로 표현하여 연속적으로 곡선을 생성한다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter06/section_06_04_02_quadratic_spline.py

6.4.3 Cubic Spline

주어진 데이터 점 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ 에 대해 각 구간 $[x_i, x_{i+1}]$ 에서 3차 다항식으로 보간한다.

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (6.4.3-1)$$

조건

1. 데이터 점 통과

$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1} \quad (6.4.3-2)$$

2. 1차 미분 연속성

$$S'_i(x_{i+1}) = S'_{i+1}(x_{i+1}) \quad (6.4.3-3)$$

3. 2차 미분 연속성

$$S''_i(x_{i+1}) = S''_{i+1}(x_{i+1}) \quad (6.4.3-4)$$

4. 자연 경계 조건 (Natural Boundary Condition)

$$S''_0(x_0) = 0, \quad S''_{n-1}(x_n) = 0 \quad (6.4.3-5)$$

6.4.4 Cubic Spline 예제

이 예제는 Cubic Spline Interpolation(3차 스플라인 보간)을 사용하여 주어진 데이터 점들을 부드럽게 연결하는 과정을 보여준다.

각 구간 $[x_i, x_{i+1}]$ 에서 3차 다항식

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

을 구성하고,

- 데이터 점 통과 조건
- 1차 미분 연속 조건
- 2차 미분 연속 조건
- 자연 경계 조건

을 적용하여 계수를 계산한다.

그 결과, 데이터 점을 정확히 통과하면서 곡선의 기울기와 곡률이 연속인 **부드러운 자연 곡선(Natural Cubic Spline)**이 생성된다.

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter06/section_06_04_04_cubic_spline.py

6.5 수치보간법들의 차이점

6.5.1 뉴턴의 분할 차분(Newton's Divided Difference Method)과 라그랑주 보간법(Lagrange Interpolation)의 동일함

수치 보간에는 대표적으로 뉴턴의 분할 차분법(Newton's Divided Difference Method)과 라그랑주 보간법(Lagrange Interpolation)이 있다. 두 방법은 표현 방식은 다르지만, 동일한 보간 다항식을 생성한다.

1. 라그랑주 보간 다항식

$$P_n(x) = \sum_{j=0}^n y_j L_j(x) \quad (6.5.1-1)$$

여기서 라그랑주 기저 다항식은

$$L_j(x) = \prod_{\substack{i=0 \\ i \neq j}}^n \frac{x - x_i}{x_j - x_i} \quad (6.5.1-2)$$

이다.

2. 뉴턴 분할 차분 다항식

$$\begin{aligned} P_n(x) = & f[x_0] + f[x_0, x_1](x - x_0) \\ & + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ & + \cdots \\ & + f[x_0, \dots, x_n] \prod_{i=0}^{n-1} (x - x_i) \end{aligned} \quad (6.5.1-3)$$

고차 분할 차분 계수는 다음과 같이 정의된다.

$$f[x_n, \dots, x_0] = \frac{f[x_n, \dots, x_1] - f[x_{n-1}, \dots, x_0]}{x_n - x_0} \quad (6.5.1-4)$$

3. 동일성의 핵심 개념

1. 뉴턴 다항식의 특징 점을 하나씩 추가하면서 점진적으로 차수를 증가시킨다.
2. 라그랑주 다항식의 특징 각 기저 다항식 $L_j(x)$ 는

$$L_j(x_i) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

을 만족하도록 구성된다.

3. 동일한 보간 다항식 생성 두 방법은 동일한 데이터 점 (x_i, y_i) 을 모두 통과하며, 결과적으로 동일한 $P_n(x)$ 을 생성한다.

결론: 뉴턴 분할 차분법과 라그랑주 보간법은 표현 형식만 다를 뿐, 수학적으로 동일한 보간 다항식을 생성한다.

6.5.2 Cubic Spline이 다른 보간법과 다른 이유

구간별 다항식 사용

고차 다항식의 Runge 현상 방지

연속성과 매끄러움 보장

6.5.3 코드 구현

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter06/section_06_05_02_cubic_spline_comparison.py

6.6 연습문제